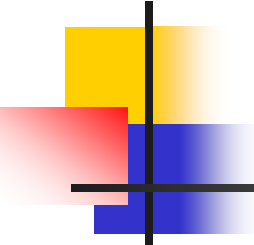


例外

プログラミング第2同演習B

2026年6月12日

高田真吾

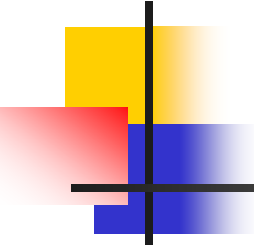


はじめに

- Javaで“やってはいけない”ことをやると、異常終了するが、その際 `XXXException` を表示することが多い.
 - 少なくとも `NullPointerException` は見ていませんか？
- 例:「0」で割った場合
 - 「`System.out.println(3/0);`」を実行すると、次のような“エラーメッセージ”が表示される.

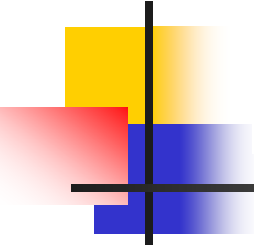
Exception in thread "main" java.lang.ArithmeticException: / by zero
at DivideEx.main(DivideEx.java:4)

- 本日は、このException(例外)を扱う.



Javaの例外の基本

- メソッドを実行中に“何か問題”が発生した場合、例外(exception)が発生する.
 - 発生する(または投げる) → throw
- その例外をきちんと捕らえることができれば、それを利用して何らかしらの対処ができる.
 - 捕らえる → catch
- Javaで例外を捕らえるための機構
→ try-catch



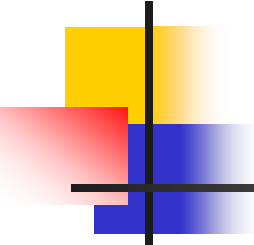
大雑把に

methodAAA()呼び出し

```
...  
try {  
    ...  
    methodAAA();  
    ...  
} catch (AAAException e) {  
    ...  
}  
...
```

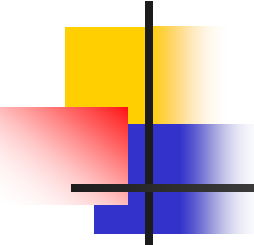
```
// 例外を投げる  
public void methodAAA()  
    throws AAAException {  
    ...  
    throw new AAAException("AAA");  
    ...  
}
```

```
// 例外はオブジェクト！  
  
public class AAAException  
    extends Exception{  
    ...  
}
```



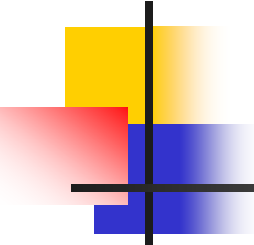
例外の処理: try-catch (1)

- ```
try { < 例外が発生するかもしれないコード > }
catch (< 例外 > e) {
 < 例外が発生したときの処理 > }
finally { < なんらかの処理 > }
```
  - try節の中: 例外が発生するかもしれないコードを書く.
  - try節を実行中に例外が発生した場合, catch節を実行.
  - try節以外の場所で例外が発生した(する可能性のある)場合, 例外の種類によって:
    - メソッドの呼び出し元に例外を自動伝播
    - そもそもコンパイルできない
- のどちらか.



## 例外の処理: try-catch (2)

- try { < 例外が発生するかもしれないコード > }  
catch ( < 例外 > e ) {  
    < 例外が発生したときの処理 > }  
finally { < なんらかの処理 > }
- 各catch節で扱う例外の種類を「<例外>」で指定する.
  - 例: 「catch (ArithmeticException e)」 { ... }
- <例外>は実はクラス名.
  - <例外>のサブクラスも扱われる.
- catch節は複数あってもOK.
- catch節を「例外ハンドラ (exception handler)」とも呼ぶ.

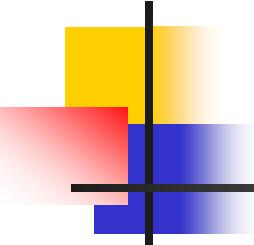


# 例外処理における実行の流れ

---

- try 節で例外が発生すると, try節内の残りのコードは実行されずに catch 節に処理が移る
- 例:  

```
try {
 System.out.println(3/0); // ここでエラー (ArithmeticException) が発生する
 System.out.println("OK");
} catch (ArithmeticException e) {
 System.out.println("ERR");
}
```
- 実行結果:  
ERR
- 今の場合, 必ずエラー(例外)が発生するので, OKを出力せず, catch節に飛ぶ.
- もし例外が発生しなかったら, OKを出力し, catch節を飛ばす.



## catch 節での処理

- 通常, エラー内容を表示して, 実行を終了する
  - ```
catch (ArithmeticException e) {  
    // 例外オブジェクトからエラーメッセージを取り出す  
    String errorMessage = e.getMessage();  
    // 標準エラー出力にエラーメッセージを表示  
    System.err.println(errorMessage);  
    // 異常終了する (引数の 1 が異常終了を表す慣例)  
    System.exit(1);  
}
```
 - 前のスライドの例のcatch節を上記にしたら, 出力結果は「/ by zero」となる.
- 次のように書くぐらいなら書かないほうがよい
 - ```
catch (ArithmeticException e) { }
```
  - エラーが起きても無視して先に進んでしまう
  - 注: try-catchを書かないとコンパイルできない場合がある.



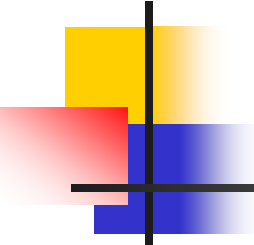
```
import java.util.*;
```

## 複数の catch 節

```
public class DivideEx4 {
 public static void main(String[] args) {
 Scanner scan = new Scanner(System.in);
 try {
 System.out.print("Enter numerator: ");
 int numerator = scan.nextInt();
 System.out.print("Enter denominator: ");
 int denominator = scan.nextInt();
 System.out.println("Result is " + numerator/denominator);
 } catch (ArithmeticException e) {
 System.err.println("Zero is invalid");
 System.exit(1);
 } catch (InputMismatchException e) {
 System.err.println("Number must be entered");
 System.exit(1);
 }
 }
}
```

- 複数のcatchがある場合、順番が重要.
- 最初にマッチする例外のcatch節が実行される.

- Scannerは標準入力から文字列を取得するために使う. 詳細は次回.
- 分母に「0」を入力すると, ArithmeticExceptionが発生.
- 分母に数字以外の文字列を入力すると, InputMismatchExceptionが発生.



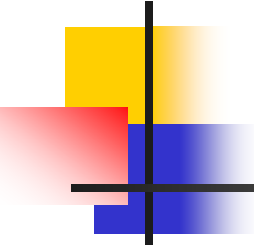
# 例外の連鎖

---

- catch節の中で例外を投げ, 元々呼び出した手続きに例外を処理させる.

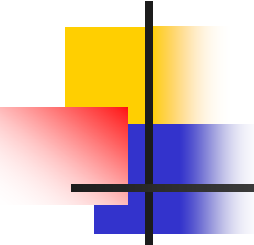
```
int[] ex = new int[]{1, 2, 3};
```

```
void printAll() {
 try {
 for (int i=1; i<4; i++) {
 System.out.println(ex[i]);
 }
 } catch (ArrayIndexOutOfBoundsException e) {
 System.err.println("Array problem!");
 throw new BadException();
 }
}
```



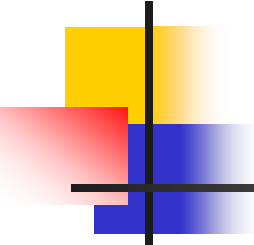
## 例外の処理: try-catch (3)

- ```
try { < 例外が発生するかもしれないコード > }  
catch ( < 例外 > e ) {  
    < 例外が発生したときの処理 > }  
finally { < なんらかの処理 > }
```
- finally節は, tryの実行を終了するとき, 必ず実行される.
- 例外が投げられても, 投げられなくても必ず実行される.
- ただし, System.exit()がその前にあると, 実行されない.
- ファイルなどの入出力を扱っているときの後処理のために使うことがある.



finally 節

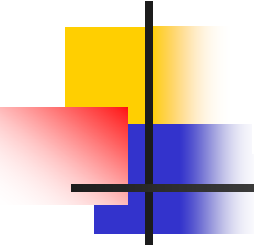
- ```
try {
 System.out.println(3/0);
 System.out.println("OK");
} catch (ArithmeticException e) {
 System.out.println("ERR");
} finally {
 System.out.println("Finally");
}
```
- try ブロックの実行中に例外が発生しても必ず Finallyが出力される.



## finally 節

---

- ```
BufferedWriter writer = null;  
try {  
    writer = new BufferedWriter(new FileWriter("foo"));  
    writer.write("This is a test program");  
    writer.newLine();  
}  
catch (IOException e) {  
    // IO エラーが起きた時の処理  
}  
finally {  
    if (writer != null) writer.close();  
}
```
- try ブロックの実行中に例外が発生しても必ず `writer.close()` が実行される
 - `FileWriter` のコンストラクタが例外を起こした時は `writer` の値が `null` のままなので, `finally` ブロックでは `writer` の値を検査している



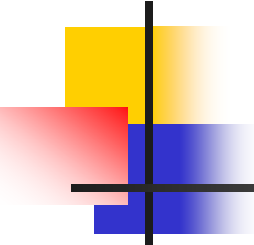
大雑把に

methodAAA()呼び出し

```
...  
try {  
    ...  
    methodAAA();  
    ...  
} catch (AAAException e) {  
    ...  
}  
...
```

```
// 例外を投げる  
public void methodAAA()  
    throws AAAException {  
    ...  
    throw new AAAException("AAA");  
    ...  
}
```

```
// 例外はオブジェクト！  
  
public class AAAException  
    extends Exception{  
    ...  
}
```



例外関連クラス

- Javaの例外の型はすべてExceptionクラスのサブクラスである.
 - 例外 (exception) も オブジェクト !
 - java.lang.Throwable クラスかそのサブクラスのインスタンス
- 例外を表す主なクラス
 - java.lang.Exception
 - 通常の例外はだいたいこのクラスのサブクラス
 - java.lang.RuntimeException
 - java.lang.Exception のサブクラスのひとつ
 - いつ起きてもおかしくない. → catchするのは難しい.
 - java.lang.Error
 - メモリを使いきったときなどの致命的な状況が発生したときに投げられる
 - try ... catch での回復処理はあきらめ, プログラム全体を終了させるのが普通

例外クラスの定義

- スーパークラス → ExceptionまたはRuntimeException
- 最低コンストラクタ(4種類)を定義する.

- 引数なし
- 文字列引数
- 他の二つは連鎖例外のため(ここでは省略)

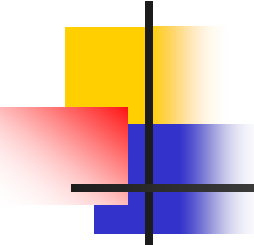
- (例)

- ```
public class NonPositiveException extends Exception {
 private static final long serialVersionUID = 1L;
 public NonPositiveException() { super(); }
 public NonPositiveException(String s) { super(s); }
}
```

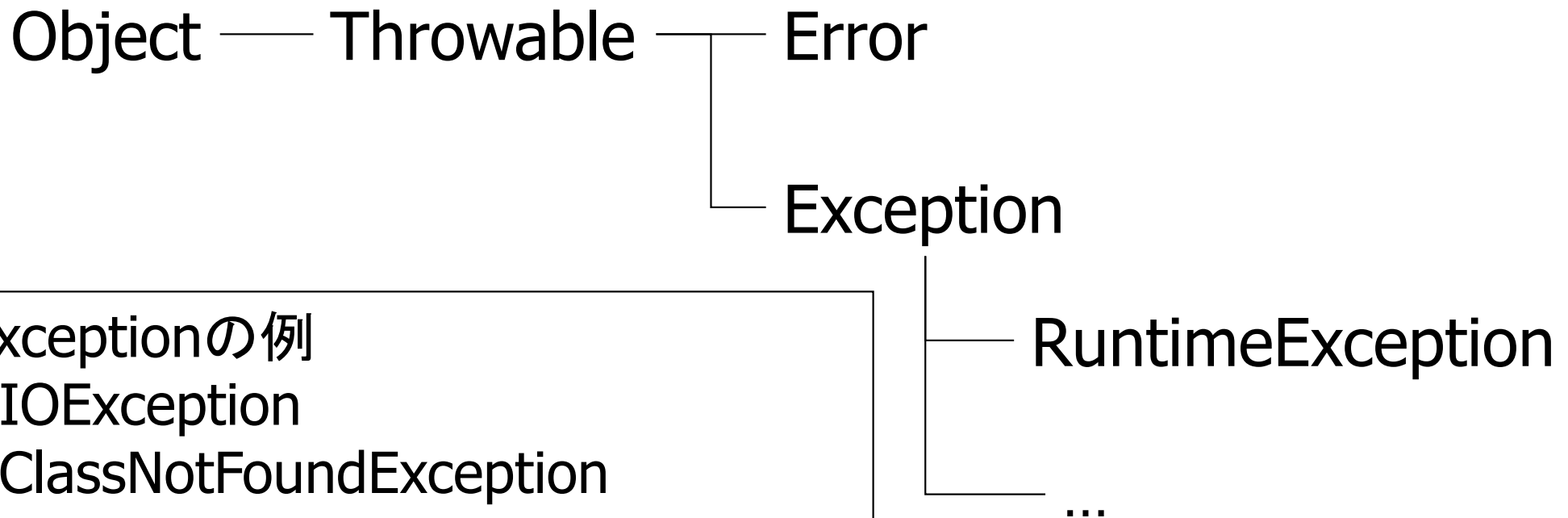
serialVersionUIDはなくても一応大丈夫だが、Warningがうるさい。

- 例外が投げられた理由を含ませたければ:  
    Exception e1 = new NonPositiveException("this is the reason");
- 理由を取り出したい場合, 「String s = e1.toString( );」とすれば, sは次の文字列を含むことになる  
    “NonPositiveException: this is the reason”





# 例外関連クラスの継承関係

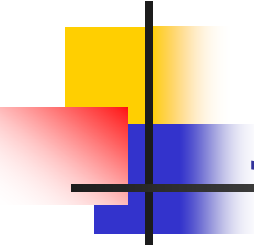


## Exceptionの例

- IOException
- ClassNotFoundException

## RuntimeExceptionの例

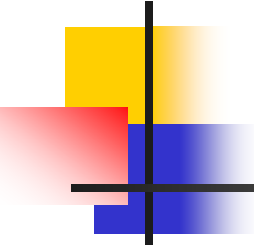
- NullPointerException
- ClassCastException
- ArrayIndexOutOfBoundsException
- InputMismatchException



# java.lang.RuntimeException クラス

---

- プログラムミスによって起きる不具合を通知する例外が多い
  - java.lang.NullPointerException
    - メソッド呼び出しや変数アクセスの対象となるオブジェクトが null であったときに投げられる
    - `String file = null;`  
`if (file.equals("abcd")) ...`
  - java.lang.ClassCastException
    - キャストに失敗したときに投げられる
    - `Object obj = new Object();`  
`Point p = (Point)obj;`
  - java.lang.ArrayIndexOutOfBoundsException
    - 配列の添え字が範囲外だった時に投げられる例外
    - `int[] a = {1, 2, 3};`  
`a[3] = 4;`



# チェック例外と非チェック例外

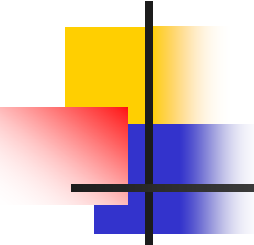
---

## ■ チェック例外

- RuntimeExceptionのサブクラスではない例外
  - 注: Errorのサブクラスは除く
- このような例外が発生する可能性のある文の場合
  - try節内に書くか,
  - そのメソッド宣言において, 投げる可能性があることを記す  
これを守らないと, コンパイルエラー.

## ■ 非チェック例外

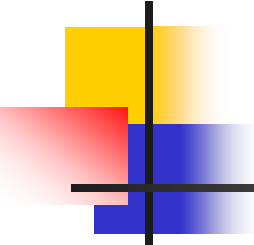
- RuntimeExceptionのサブクラス
- 例外処理の記述はあってもなくても構わない.



# 例外の処理: 非チェック例外

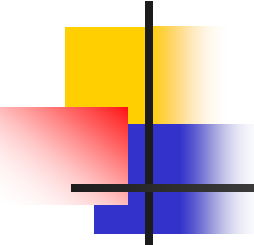
- 非チェック例外はいつ起きてもおかしくない.
  - RuntimeExceptionのサブクラス
- そのため:
  - 非チェック例外をcatchするのは難しい.
  - どこから来るのか特定するのは難しい.
- ```
try {  
    x = y[n];  
    i = Arrays.search(z, x);  
} catch (ArrayIndexOutOfBoundsException e) {  
    < 例外処理のコード >  
}
```

この場合, どこで例外が発生したのか?



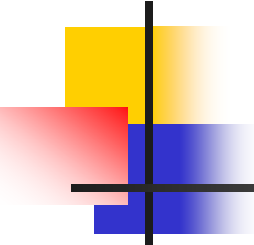
～論争(脱線だけれど知っておくとよい)～ チェック例外の是非(1)

- Javaではチェック例外と非チェック例外があることを知ってプログラミングする必要がある.
 - チェック例外は必ず対応しなければならないが, 非チェック例外は(極端に言うと)無視できる.
- チェック例外は批判的になっており, いろいろな問題が指摘されている.
 - 例: コードが読みにくなる.



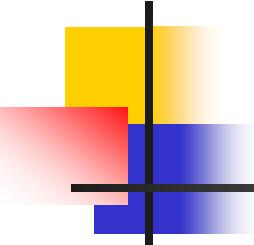
～論争(脱線だけれど知っておくとよい)～ チェック例外の是非 (2)

- プログラム中で、例外処理に関するコードの割合が高くなると読みにくなる.
- 慣れてくると、「catchは無視しちゃおう！」
 - catch中に何もしていない
 - エラーメッセージを出力するだけ
- でも、そんなことをしたら、本当に重要な例外処理コードを見過ごすかもしれない.



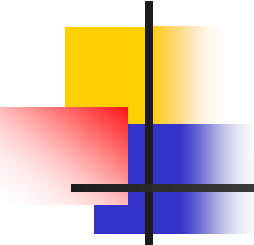
～論争(脱線だけれど知っておくとよい)～ チェック例外の是非 (3)

- コード量を減らすために、catch節を減らしても、理解しにくくなる.
- あるメソッドが5種類の例外を投げる可能性がある場合、本来それぞれに対してcatch節(5つ!)を用意しなければならない.
- スーパークラス Exception でまとめてしまえ!
 - どうせcatch節中に何もしない、もしくはエラーメッセージを出力するだけ.
- でも、そんなことをしたら、どういう例外が来るのかわからなくなるが、それでよいのか？



～論争(脱線だけれど知っておくとよい)～ チェック例外の是非 (4)

- 例外の(自動)伝播は, メソッドの抽象レベルとの不整合が発生し, コードをわかりにくくする.
- JDBC APIのほとんどのメソッドは, SQLExceptionを投げる可能性がある.
- しかし, SQLExceptionを捕らえたら, どういう意味で発生したのか, すぐにはわからない.
 - Exceptionを捕らえたのと似ている.
 - 例外が伝播したら, SQLを使っているか否かについてはどうでもよいメソッドに到達するかもしれない.
→ 実装の詳細を見せてしまう.
- 例外の連鎖を利用すると, 問題を緩和できる.
 - 例外を投げなおすとき, 抽象レベルに適した名前にする.



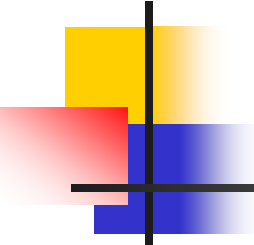
大雑把に

methodAAA()呼び出し

```
...  
try {  
    ...  
    methodAAA();  
    ...  
} catch (AAAException e) {  
    ...  
}  
...
```

```
// 例外を投げる  
public void methodAAA()  
    throws AAAException {  
    ...  
    throw new AAAException("AAA"),  
    ...  
}
```

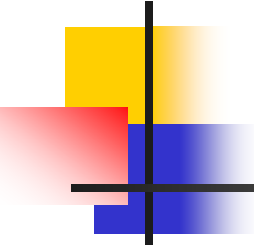
```
// 例外はオブジェクト！  
  
public class AAAException  
    extends Exception{  
    ...  
}
```



例外を投げる

- throw文を使って、例外を投げる.
 - if (n <= 0)
 throw new NonPositiveException("Num.fact");
 - 引数には、通常、ユーザにどういう問題が発生したかを教えられる文字列にする.
 - 例: クラス名とメソッド名にすれば、どこで発生したか調べられる.
 - throwしたら、メソッドは終了する.
- throw文がある場合、非チェック例外か否かによって、メソッドの宣言部に例外が発生することを明記する必要がある.
- (例)
 - public boolean fact (int n) **throws** NonPositiveException {
 if (n <= 0)
 throw new NonPositiveException("Num.fact");
 return true;
}

throws なのか throw なのかに注意！



いつthrowsを書かないといけないのか？

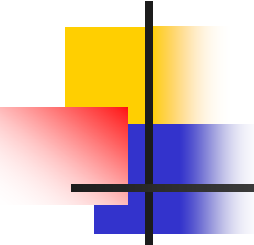
```
import java.io.IOException;
```

```
public class Temp {  
    public void methodA() {  
        throw new NumberFormatException();  
    }  
}
```

```
public void methodB() {  
    throw new IOException();  
}
```

```
public static void main(String[] args) {  
    Temp app = new Temp();  
    app.methodA();  
    app.methodB();  
}
```

- 非チェック例外はthrowしてもメソッド宣言で書かなくてもよい.
 - methodAは問題なし.
- チェック例外はthrowしたらメソッド宣言で書かないといけない.
 - methodBは問題あり.



例外を捕捉しない？

- チェック例外が発生する可能性がある場合
 - try節内に書くか,
 - メソッドが例外を投げる可能性があることを宣言する必要がある
 - 例:
// メソッド causeException が例外 IOException を投げることがあると宣言
static void causeException() throws IOException {

 ...
 in.readLine(); // ここで例外が発生するかも
 ... // でも catch はしていない
}
static void callCauseException() {
 try {
 causeException(); // causeException で例外が発生すると
 // ここで例外が発生する
 } catch (IOException e) {
 // メソッド causeException の中で発生した例外がここで捕捉される
 }
 ...
}

■ throws 節に書いたクラスおよびそのサブクラスの例外が投げられる
■ throws 節にはカンマ (,) で区切って複数の例外を並べることができる

例外発生時のメッセージ

(注)Javaのバージョンによって、メッセージの細かいところが異なる

- (非チェック例外など)例外をキャッチしないと次のようなメッセージが表示される.

例外の種類

何が表示されるかは例外による

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 4
    at Temp.check(Temp.java:7)
    at Temp.main(Temp.java:12)
```

例外が発生した場所

この例では、Tempクラスのmainメソッド内(Temp.javaファイルの12行目)でTempクラスのcheckメソッドを呼び出し、そこで(Temp.javaの7行目で)例外が発生している。